Full Stack Developer (React.js & Node.js) Assessment:

Estimated Completion Time: 8-12 hours. We value quality, clean code, and thoughtful implementation over rushed features. **Submission Deadline:** Please submit your completed assessment within 7 days from the date you receive this brief. Late submissions may not be considered unless prior arrangements are made.

Assessment Brief: Build a Task Management Module

Module Overview: Develop a secure web application for managing tasks with assignment capabilities. Users should be able to register, authenticate, create tasks, assign them to other users, and perform CRUD (Create, Read, Update, Delete) operations. Task visibility is restricted: Users can only see and manage tasks assigned to them. Task statuses are dynamically derived based on completion and due date comparisons (e.g., Missed/Late for overdue, done for completed, Due Today for tasks due on the current day). This module should be built as a standalone app but designed with modularity in mind for easy integration into larger systems (e.g., linking tasks to CRM customer records or HR employee profiles in future iterations).

Key User Stories (Interpret creatively to build an intuitive experience):

- As a new user (e.g., an employee or manager), I can sign up with a unique email and secure password.
- As a user, I can log in and log out, with sessions maintained securely.
- As a logged-in user, I can create a task with fields: title (required), description (optional), due date (required, date format), priority (optional: low/medium/high), and assign it to another user (by email; assume the assignee must exist in the system).
- As a logged-in user, I can view a dashboard listing only the tasks assigned to me, sorted by due date (ascending by default), with filters for derived status (e.g., Missed/Late, Done, Due Today) or priority.
- As a logged-in user, I can edit task details (if assigned to me), delete a task (if assigned to me or if I'm the creator), or toggle its completion status.
- The system dynamically computes task status based on due date and completion:
    - Done: If marked as completed.
    - Missed/Late: If not completed and due date has passed.
    - Due Today: If not completed and due date is today.
    - (Optional: Upcoming for future due dates-add if it enhances UX.)
- The system ensures data privacy: Users can only access tasks assigned to them (no visibility into others' tasks, even if created by them unless self-assigned).
- As a task creator, I can assign/reassign tasks to users, but once assigned, only the assignee can view/edit/complete it (creators can still reassign or delete).
- Handle edge cases gracefully, such as invalid inputs, expired sessions, non-existent assignees, or network errors, with user-friendly feedback (e.g., "Assignee email not found").

Non-Functional Requirements:

- **Security:** Implement authentication (e.g., session-based or JWT), password hashing (e.g., using bcrypt in Node.js), input validation/sanitization to prevent injection attacks, XSS, etc. Ensure assignment logic checks for valid users. Assume integration potential with WhatsApp for notifications (e.g., structure APIs to support future webhooks for task assignments or due date reminders).
- **Performance & Scalability:** Optimize API responses, minimize React re-renders, and design database queries for efficiency (e.g., index due dates for sorting). Structure code modularly for extensibility (e.g., adding LMS quiz tasks or ERP inventory links later).
- **Usability:** Create a modern, responsive UI (mobile-first) that's visually appealing and intuitive- draw inspiration from business tools like ERP dashboards. Display derived statuses prominently (e.g., color-coded: red for Missed/Late, yellow for Due Today, green for Done).
- **Error Handling:** Provide clear, professional messages (e.g., "Task assignment failed: User not found") and logging for debugging.
- **Best Practices:** Write clean, maintainable code with comments. Use version control (Git) to demonstrate logical commit history. Handle date comparisons accurately (consider time zones if relevant, but local time is fine for simplicity).

Technical Specifications

Frontend (React.js):

- Bootstrap with Create React App, Vite, or similar.
- Use React Router for navigation (e.g., routes for login, signup, task dashboard).
- Manage state with hooks (useState, useEffect) and optionally Context API or Redux for authentication, task data, and status computations.
- Consume backend APIs via fetch or Axios; implement assignee search/lookup if needed (e.g., API to validate emails).
- Compute and display dynamic statuses client-side based on due date.
- Bonus: Add subtitle UX enhancements like loading spinners, date pickers, assignee autocomplete, or real-time status updates.

Backend (Node.js):

- Use Node.js with Express.js; *frameworks like NestJS are optional* but appreciated if they enhance efficiency.
- Set up a database (e.g., MongoDB or MySQL) with models/tables: users (id, email, password_hash) and tasks (id, creator_id, assignee_id, title, description, due_date, priority, is_completed).
- Ensure relationships/foreign keys for integrity.
- Expose RESTful APIs (e.g., POST/api/auth/signup, POST /api/auth/login, GET/POST/PUT/DELETE/api/tasks, POST/api/tasks/assign for assignment).
- Use secure database queries (e.g., using an ORM/ODM like Mongoose or Sequelize); queries should filter tasks by assignee_id = current_user_id.
- Handle CORS for frontend integration; validate assignees exist before assignment.

Integration & Deployment:

- Ensure seamless API communication between frontend and backend.
- Include environment setup for local testing (e.g., via Docker, nodemon, or .env files).

Submission Instructions

- Submit by emailing the repository link to development.team@innovation-gate.ae before the deadline: within 7 days from receipt.

Evaluation Criteria

Your submission will be reviewed based on:

- **Functionality & Completeness (30%):** Full implementation of user stories, including assignment and dynamic statuses, with minimal bugs.
- **Code Quality & Maintainability (25%):** Readability, modularity, and adherence to best practices.
- **Frontend Proficiency (20%):** Effective React usage, responsive design, creative Ul interpretation, and status handling.
- **Backend Proficiency (20%):** Secure, efficient Node.js and database handling, with proper assignment logic.
- **Innovation & Alignment (5%):** How well the module fits our ERP/CRM/HR/LMS focus (e.g., scalable design for integrations like task assignments in HR).

We appreciate your time and look forward to reviewing your work by the deadline. If selected, we'll discuss your approach in a follow-up technical interview.